用内存顺序实现内存模型

Posted on 2023-10-28 12:41:38 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2XNSZPZONCk5COb8XWbUJEPnI4o)

前情回顾

前文我们介绍了六种内存顺序，以及三种内存模型，本文通过代码示例讲解六种内存顺序使用方法，并实现相应的内存模型。

memory_order_seq_cst

`memory_order_seq_cst` 代表全局一致性顺序，可以用于 `store`，`load` 和 `read-modify-write` 操作, 实现 `sequencial consistent` 的顺序模型. 在这个模型下, 所有线程看到的所有操作都有一个一致的顺序, 即使这些操作可能针对不同的变量, 运行在不同的线程.

我们看一下之前写的代码

```
1. std::atomic<bool> x, y;
2. std::atomic<int> z;
3.
4. void write_x_then_y() {
5.     x.store(true, std::memory_order_relaxed);  // 1
6.     y.store(true, std::memory_order_relaxed);  // 2
7. }
8.
9. void read_y_then_x() {
10.     while (!y.load(std::memory_order_relaxed)) { // 3
11.         std::cout << "y load false" << std::endl;
12.     }
13.
14.     if (x.load(std::memory_order_relaxed)) { //4
15.         ++z;
16.     }
17.
18. }
19.
20. void TestOrderRelaxed() {
21.
22.     std::thread t1(write_x_then_y);
23.     std::thread t2(read_y_then_x);
24.     t1.join();
25.     t2.join();
26.     assert(z.load() != 0); // 5
27. }
```

上面的代码 `load` 和 `store` 都采用的是 `memory_order_relaxed` 。线程t1按次序执行1和2，但是线程t2看到的可能是y为true,x为false。进而导致 `TestOrderRelaxed` 触发断言z为0.
如果换成 `memory_order_seq_cst` 则能保证所有线程看到的执行顺序是一致的。

```
1.
2. void write_x_then_y() {
3.     x.store(true, std::memory_order_seq_cst);  // 1
4.     y.store(true, std::memory_order_seq_cst);  // 2
5. }
6.
7. void read_y_then_x() {
8.     while (!y.load(std::memory_order_seq_cst)) { // 3
9.         std::cout << "y load false" << std::endl;
10.    }
11.
12.    if (x.load(std::memory_order_seq_cst)) { //4
13.        ++z;
14.    }
15.
16. }
17.
18. void TestOrderSeqCst() {
19.
20.    std::thread t1(write_x_then_y);
21.    std::thread t2(read_y_then_x);
22.    t1.join();
23.    t2.join();
24.    assert(z.load() != 0); // 5
25. }
```

上面的代码x和y采用的是 `memory_order_seq_cst` , 所以当线程t2执行到3处并退出循环时我们可以断定y为true，因为是全局一致性顺序，所以线程t1已经执行完2处将y设置为true，那么线程t1也一定执行完1处代码并对t2可见，所以当t2执行至4处时x为true，那么会执行z++保证z不为零，从而不会触发断言。

实现 sequencial consistent 模型有一定的开销. 现代 CPU 通常有多核, 每个核心还有自己的缓存. 为了做到全局顺序一致, 每次写入操作都必须同步给其他核心. 为了减少性能开销, 如果不需要全局顺序一致, 我们应该考虑使用更加宽松的顺序模型.

memory_order_relaxed

memory_order_relaxed 可以用于 store, load 和 read-modify-write 操作, 实现 relaxed 的顺序模型.
前文我们介绍过这种模型下, 只能保证操作的原子性和修改顺序 (modification order) 一致性,

无法实现 synchronizes-with 的关系。

```
1. void TestOrderRelaxed() {
2.     std::atomic<bool> rx, ry;
3.
4.     std::thread t1([&]() {
5.         rx.store(true, std::memory_order_relaxed); // 1
6.         ry.store(true, std::memory_order_relaxed); // 2
7.         });
8.
9.
10.    std::thread t2([&]() {
11.        while (!ry.load(std::memory_order_relaxed)); //3
12.        assert(rx.load(std::memory_order_relaxed)); //4
13.        });
14.
15.    t1.join();
16.    t2.join();
17. }
```

上面的代码在一定程度上会触发断言。因为线程t1执行完1，2之后，有可能2操作的结果先放入内存中被t2看到，此时t2执行退出3循环进而执行4，此时t2看到的rx值为false触发断言。

我们称2和3不构成同步关系， 2 “ not synchronizes with “ 3

如果能保证2的结果立即被3看到, 那么称 2 “synchronizes with “ 3。

如果2 同步于 3还有一层意思就是 如果在线程t1 中 1 先于 2(sequence before)， 那么 1先行于3。那我们可以理解t2执行到3处时，可以获取到t1执行1操作的结果，也就是rx为true.

t2线程中3先于4（sequence before），那么1 操作先行于 4. 也就是1 操作的结果可以立即被4获取。进而不会触发断言。

怎样保证2 同步于 3 是解决问题的关键， 我们引入 Acquire-Release 内存顺序。

Acquire-Release

在 acquire-release 模型中, 会使用 memory_order_acquire, memory_order_release 和 memory_order_acq_rel 这三种内存顺序. 它们的用法具体是这样的:

对原子变量的 load 可以使用 memory_order_acquire 内存顺序. 这称为 acquire 操作.

对原子变量的 store 可以使用 memory_order_release 内存顺序. 这称为 release 操作.

read-modify-write 操作即读 (load) 又写 (store), 它可以使用 memory_order_acquire, memory_order_release 和 memory_order_acq_rel:

1. 如果使用 memory_order_acquire, 则作为 acquire 操作;
2. 如果使用 memory_order_release, 则作为 release 操作;
3. 如果使用 memory_order_acq_rel, 则同时为两者.

Acquire-release 可以实现 synchronizes-with 的关系. 如果一个 acquire 操作在同一个原子变量上读取到了一个 release 操作写入的值, 则这个 release 操作 “synchronizes-with” 这个 acquire 操作.

我们可以通过Acquire-release 修正 TestOrderRelaxed函数以达到同步的效果

```
1. void TestReleaseAcquire() {
2.     std::atomic<bool> rx, ry;
3.
4.     std::thread t1([&]() {
5.         rx.store(true, std::memory_order_relaxed); // 1
6.         ry.store(true, std::memory_order_release); // 2
7.         });
8.
9.
10.    std::thread t2([&]() {
11.        while (!ry.load(std::memory_order_acquire)); //3
12.        assert(rx.load(std::memory_order_relaxed)); //4
13.        });
14.
15.    t1.join();
16.    t2.join();
17. }
```

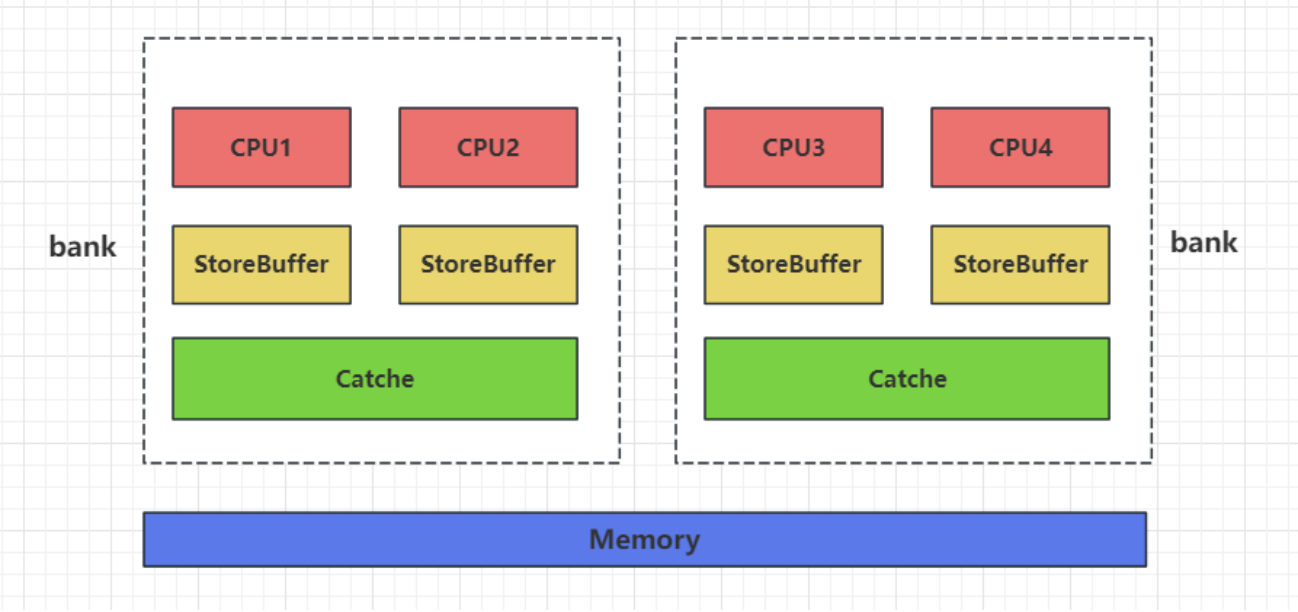
上面的例子中我们看到 `ry.store` 使用的是 `std::memory_order_release` ， `ry.load` 使用的是 `std::memory_order_relaxed` .

t1执行到2将ry 设置为true, 因为使用了Acquire-release 顺序， 所以 t2 执行到3时读取ry为true， 因此2和3 可以构成同步关系。

又因为单线程t1内 1 sequence before 2,所以1 happens-before 3.
因为单线程t2内 3 sequence before 4. 所以 1 happens-before 4.

可以断定4 不会触发断言。

我们从cpu结构图理解这一情景



到此大家一定要记住仅 Acquire-release能配合达到 synchronizes-with效果，再就是memory_order_seq_cst可以保证全局顺序唯一，其他情况的内存顺序都能保证顺序，使用时需注意。

Acquire-release 的开销比 sequencial consistent 小. 在 x86 架构下, memory_order_acquire 和 memory_order_release 的操作不会产生任何其他的指令, 只会影响编译器的优化: 任何指令都不能重排到 acquire 操作的前面, 且不能重排到 release 操作的后面; 否则会违反 acquire-release 的语义. 因此很多需要实现 synchronizes-with 关系的场景都会使用 acquire-release.

Release sequences

我们再考虑一种情况，多个线程对同一个变量release操作，另一个线程对这个变量acquire，那么只有一个线程的release操作喝这个acquire线程构成同步关系。

看下面的代码：

```
1. void ReleasAcquireDanger2() {
2.     std::atomic<int> xd{0}, yd{ 0 };
3.     std::atomic<int> zd;
4.
5.     std::thread t1([&]() {
6.         xd.store(1, std::memory_order_release);  // (1)
7.         yd.store(1, std::memory_order_release); //  (2)
8.         });
9.
10.    std::thread t2([&]() {
11.        yd.store(2, std::memory_order_release);  // (3)
12.        });
13.
14.
15.    std::thread t3([&]() {
16.        while (!yd.load(std::memory_order_acquire)); //  (4)
17.        assert(xd.load(std::memory_order_acquire) == 1); // (5)
18.        });
19.
20.    t1.join();
21.    t2.join();
22.    t3.join();
23. }
```

我们可以看到t3在yd为true的时候才会退出，那么导致yd为true的有两种情况，一种是1，另一种是2， 所以5处可能触发断言。

并不是只有在 acquire 操作读取到 release 操作写入的值时才能构成 synchronizes-with 关系. 为了说这种情况, 我们需要引入 release sequence 这个概念.

针对一个原子变量 M 的 release 操作 A 完成后, 接下来 M 上可能还会有一连串的其他操作. 如果这一连串操作是由

1. 同一线程上的写操作
2. 任意线程上的 read-modify-write 操作
   这两种构成的, 则称这一连串的操作为以 release 操作 A 为首的 release sequence. 这里的写操作和 read-modify-write 操作可以使用任意内存顺序.

如果一个 acquire 操作在同一个原子变量上读到了一个 release 操作写入的值, 或者读到了以这个 release 操作为首的 release sequence 写入的值, 那么这个 release 操作 “synchronizes-with” 这个 acquire 操作.

看下面的代码

```
1. void ReleaseSequence() {
2.     std::vector<int> data;
3.     std::atomic<int> flag{ 0 };
4.
5.     std::thread t1([&]() {
6.         data.push_back(42);  //(1)
7.         flag.store(1, std::memory_order_release); //(2)
8.         });
9.
10.    std::thread t2([&]() {
11.        int expected = 1;
12.        while (!flag.compare_exchange_strong(expected, 2, std::memory_order_relaxed)) // (3)
13.            expected = 1;
14.        });
15.
16.    std::thread t3([&]() {
17.        while (flag.load(std::memory_order_acquire) < 2); // (4)
18.        assert(data.at(0) == 42); // (5)
19.        });
20.
21.    t1.join();
22.    t2.join();
23.    t3.join();
24. }
```

我们考虑t3要想退出首先flag要等于2，那么就要等到t2将flag设置为2，而flag设置为2又要等到t1将flag设置为1. 所以我们捋一下顺序 `2->3->4`

t1中操作2是release操作，以2为开始，其他线程(t2)的读改写在release操作之后，我们称之为release sequence， t3要读取release sequence写入的值，所以我们称t1的release操作“synchronizes with “ t3的 acquire 操作。

memory_order_consume

`memory_order_consume` 其实是 `acquire-release` 模型的一部分, 但是它比较特殊, 它涉及到数据间相互依赖的关系. 就是前文我们提及的 `carries dependency` 和 `dependency-ordered before` .

我们复习一下

如果操作 a “sequenced-before” b, 且 b 依赖 a 的数据, 则 a “carries a dependency into” b. 一般来说, 如果 a 的值用作 b 的一个操作数, 或者 b 读取到了 a 写入的值, 都可以称为 b 依赖于 a

```
1. p++;   // (1)
2. i++;   // (2)
3. p[i]   // (3)
```

(1) “sequenced-before” (2), (2) “sequenced-before” (3), 而(1)和(2)的值作为(3)的下表运算符[]的操作数。

我们可以称(1) “carries a dependency into “ (3), (2) “carries a dependency into “ (3), 但是(1)和(2)不是依赖关系。

memory_order_consume 可以用于 load 操作. 使用 memory_order_consume 的 load 称为 consume 操作. 如果一个 consume 操作在同一个原子变量上读到了一个 release 操作写入的值, 或以其为首的 release sequence 写入的值, 则这个 release 操作 “dependency-ordered before” 这个 consume 操作.

看下面这个例子

```
1. void ConsumeDependency() {
2.     std::atomic<std::string*> ptr;
3.     int data;
4.
5.     std::thread t1([&]() {
6.         std::string* p = new std::string("Hello World"); // (1)
7.         data = 42; // (2)
8.         ptr.store(p, std::memory_order_release); // (3)
9.         });
10.
11.    std::thread t2([&]() {
12.        std::string* p2;
13.        while (!(p2 = ptr.load(std::memory_order_consume))); // (4)
14.        assert(*p2 == "Hello World"); // (5)
15.        assert(data == 42); // (6)
16.        });
17.
18.    t1.join();
19.    t2.join();
20. }
```

t2执行到(4)处时，需要等到ptr非空才能退出循环，这就依赖t1执行完(3)操作。

因此(3) “dependency-ordered before” (4), 根据前文我们介绍了dependency等同于synchronizes ，所以(3) “inter-thread happens-before”. （4）

因为(1) “sequenced before” (3), 所以(1) “happens-before “ (4)

因为(4) “sequenced before” (5), 所以(1) “happens-before “ (5)

所以(5)处断言也不会触发。

因为(2) 和(3)不构成先行关系，所以(6)处断言可能触发。

单例模式改良

还记得我们之前用智能指针双重检测方式实现的单例模式吗？我当时说过是存在线程安全问题的，看看下面这段单例模式

```
1. //利用智能指针解决释放问题
2. class SingleAuto
3. {
4. private:
5.     SingleAuto()
6.     {
7.     }
8.     SingleAuto(const SingleAuto&) = delete;
9.     SingleAuto& operator=(const SingleAuto&) = delete;
10. public:
11.     ~SingleAuto()
12.     {
13.         std::cout << "single auto delete success " << std::endl;
14.     }
15.     static std::shared_ptr<SingleAuto> GetInst()
16.     {
17.         // 1 处
18.         if (single != nullptr)
19.         {
20.             return single;
21.         }
22.         // 2 处
23.         s_mutex.lock();
24.         // 3 处
25.         if (single != nullptr)
26.         {
27.             s_mutex.unlock();
28.             return single;
29.         }
30.         // 4处
31.         single = std::shared_ptr<SingleAuto>(new SingleAuto);
32.         s_mutex.unlock();
33.         return single;
34.     }
35. private:
36.     static std::shared_ptr<SingleAuto> single;
37.     static std::mutex s_mutex;
38. };
```

我们写一段代码测试一下

```
1. std::shared_ptr<SingleAuto> SingleAuto::single = nullptr;
2. std::mutex SingleAuto::s_mutex;
3.
4. void TestSingle() {
5.     std::thread t1([]() {
6.         std::cout << "thread t1 singleton address is 0X: " << SingleAuto::GetInst() << std::endl;
7.         });
8.
9.     std::thread t2([]() {
10.         std::cout << "thread t2 singleton address is 0X: " << SingleAuto::GetInst() << std::endl;
11.         });
12.
13.     t2.join();
14.     t1.join();
15. }
```

虽然可以正常输出两次的地址都是同一个，但是我们的单例会存在安全隐患。
1处和4处代码存在线程安全问题，因为4处代码在之前的文章中我谈过，new一个对象再赋值给变量时会存在多个指令顺序

第一种情况

```
1. 1 为对象allocate一块内存空间
2. 2 调用construct构造对象
3. 3 将构造到的对象地址返回
```

第二种情况

```
1. 1 为对象allocate一块内存空间
2. 2 先将开辟的空间地址返回
3. 3 调用construct构造对象
```

如果是第二种情况，在4处还未构造对象就将地址返回赋值给single，而此时有线程运行至1处判断single不为空直接返回单例实例，如果该线程调用这个单例的成员函数就会崩溃。

为了解决这个问题，我们可以通过内存模型来解决

```
1. //利用智能指针解决释放问题
2. class SingleMemoryModel
3. {
4. private:
5.     SingleMemoryModel()
6.     {
7.     }
8.     SingleMemoryModel(const SingleMemoryModel&) = delete;
9.     SingleMemoryModel& operator=(const SingleMemoryModel&) = delete;
10. public:
11.     ~SingleMemoryModel()
12.     {
13.         std::cout << "single auto delete success " << std::endl;
14.     }
15.     static std::shared_ptr<SingleMemoryModel> GetInst()
16.     {
17.         // 1 处
18.         if (_b_init.load(std::memory_order_acquire))
19.         {
20.             return single;
21.         }
22.         // 2 处
23.         s_mutex.lock();
24.         // 3 处
25.         if (_b_init.load(std::memory_order_relaxed))
26.         {
27.             s_mutex.unlock();
28.             return single;
29.         }
30.         // 4处
31.         single = std::shared_ptr<SingleMemoryModel>(new SingleMemoryModel);
32.         _b_init.store(true, std::memory_order_release);
33.         s_mutex.unlock();
34.         return single;
35.     }
36. private:
37.     static std::shared_ptr<SingleMemoryModel> single;
38.     static std::mutex s_mutex;
39.     static std::atomic<bool> _b_init ;
40. };
41.
42. std::shared_ptr<SingleMemoryModel> SingleMemoryModel::single = nullptr;
```

```
43. std::mutex SingleMemoryModel::s_mutex;
44. std::atomic<bool> SingleMemoryModel::_b_init = false;
```

然后我们测试

```
1. void TestSingleMemory() {
2.     std::thread t1([]() {
3.         std::cout << "thread t1 singleton address is 0x: " << SingleMemoryModel::GetInst() << st
   d::endl;
4.         });
5.
6.     std::thread t2([]() {
7.         std::cout << "thread t2 singleton address is 0x: " << SingleMemoryModel::GetInst() << st
   d::endl;
8.         });
9.
10.    t2.join();
11.    t1.join();
12. }
```

也可以看到输出的地址一致，但是我们这个改进的版本防止了线程安全问题。

总结

本文介绍了如何通过内存顺序实现内存模型，以及优化了单例模式。

源码链接

https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day11-AcquireRelease (https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day11-AcquireRelease)

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

浏览(2188) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(552) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)